

DATA STRUCTURE AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	VARSHINI CHILLARA
ROLL NO	AV.SC.U4CSE25105
YEAR/SEM/SECTION	2nd YEAR/1/B
CREDITS	05

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	DATE
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	5-11	14-07-26
2.	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	12-16	21-07-26
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	17-21	01-08-26

INDEX

S.NO	TITLE	PAGE NO	DATE
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	22-25	11-08-26
5.	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	26-59	18-08-26
6	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	60-67	25-08-26

INDEX

S.NO	TITLE	PAGE NO	DATE
7.	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	68-85	22-09-26
8.	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	86-89	29-09-26
9.	**WEEK 9:** **BinarySearch Trees** Pythonprogramtoimplement Binary Search Tree (BST) operations.	90-94	06-10-26
10.	**WEEK 10:** **AVL TREE** Write a Python program to implement an AVL Tree with all its operations.	95-104	13-10-26
11.	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	105-108	27-10-26

LAB – 1

1.AIM : A rocketlaunch system displays a countdown before launch.Instead of using aloopimplement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
for i in range(n,-1,-1):
if i==0:
print("launch")
else:
 print(i)
n=int (input ("Enter countdown number:"))
launch(n)
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/countdowna.py ========
Enter countdown number: 7
7
6
5
4
3
2
1
launch
```

EXPLANATION: This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

Ituses a recursive function named countdown().

•The function calls itself with the value n-1.
•The recursion continues until n becomes 0.
•When n reaches 0, the program displays "Launch!".

ERRORTABLE

:	Errors Indentation	Rectification
S.No 1.	Error	Use 3 spaces for each level of indentation
2.	Type Error	Convert the input to an integer using n=int(input(“Enter no,of seconds to launch:”)).

1.B.AIM :

Thebankscalculatethegrowth of an investment using compoundinterest;theformula requires calculating powers'.Whichcanbeimplemented recursively.Write a recursiveprogramtocalculate p^n(power),where p=principal growthfactor,n=numberof years.

PROGRAM:

```
def power (p, n):
 if n == 0:
 return 1
 else:
return p * power (p, n - 1)
p=float(input("Enterprincipal/base value: "))
n=int(input("Enternumber of years/powers: "))
 result = power (p, n)
print("Power =", result)
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/interest.py ==========
Enter principal/base value: 2000
Enter number of years/powers: 8
Power = 2.56e+26
```

EXPLANATION:

The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).

The recursion stops when n becomes 0, returning 1, which is the base case.
The final result is displayed after all recursive calls are completed.

•Prepresents the principal/growth amount.
•nrepresents the number of years.
•The user enters the values of P and n. •The growth() function calls itself with n-1.
•When n becomes 0, the function returns 1.
•The result is calculated by repeatedly multiplying P.

ERROR TABLE:

S.No	***Errors*** Type	***Rectification*** Convert
1	Error	the input to float or int using float(input()) or (input()). Change the recursive
2.	Incorrect output If the recursive call is written as power(p, n+1) instead of power(p, n-1).	call to power(p,n-1) so that n decreases toward the base case.

1.c aim

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
defsearch(lst, key, index):
    ifindex >= len(lst):
        return False

    iflst[index] == key:
        return True

    return search(lst, key, index + 1)


employees = [101, 102, 103, 104, 105]

key=int(input("Enter Employee ID to search: "))

ifsearch(employees, key, 0):
    print(f"Employee ID found: {key}")
else:
    print("Employee ID Not Found")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/employee.py ==========
Enter Employee ID to search: 102
Employee ID found: 102
```

EXPLANATION:

The recursive function checks each employee ID in the list one by one, starting from the first element.
Ifthe required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
Ifthe entire list is searched without finding the ID, the function returns False.

ERROR TABLES:

S.no	Errors	Rectification
1.	Name Error if the list name is misspelled(e.g…employee instead of employees)’	Use the correct variable consistently throughout the program.
2.	Syntax Error due to a missing colon(:) after an if statement or function definition.	Add the missing colon(:) after if,else,or def.

4.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using factorial formula(n!).develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
deffactorial(n):
    ifn==0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


num=int(input("Enter a number: "))

ifnum<0:
    print("Factorial does not exist for negative numbers.")
else:
    print("Factorial =", factorial(num))
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/factorial.py ==========
Enter a number: 8
Factorial = 40320
```

EXPLANATION:

•The program calculates the factorial of a given number.
•Factorial is represented as n!.
•The factorial() function calls itself with n-1.
•If n is 1, the function returns 1.
•Otherwise, the current value of n is multiplied by the factorial of n-1.
•Negative or invalid values are handled by displaying a suitable message.

ERROR TABLE:

SI.No	Errors	***Rectification***
1	Indentation Error if the statements inside the function are not properly indented	Maintain proper indentations(usually 4 spaces) inside the function
2	Type Error if a decimal (e.g,5.5) is entered for factorial calculation	***Enter only a whole number or convert the input using int()***

5.AIM

Write a recursive python program to print the first nterms of the Fibonacci series

PROGRAM:

```
ifn==0:
deffibonacci(n):
   return 0
elifn== 1:
  return 1
     elsereturn fibonacci(n - 1) + fibonacci(n - 2)


n=int(input("Enter the number of terms: "))

print("Fibonacci Series:")

foriinrange(n):
    print(fibonacci(i), end=" ")
```

OUTPUT

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/FIONAC
CI.PY
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
```

EXP LAN AT ION :

• The program prints the first n terms of the Fibonacci series. • The fib() function calculates Fibonacci numbers recursively. • The first term is 0.

• The second term is 1.

• Each subsequent term is calculated by adding the previous two terms.

• The print_fib() function is used to print the Fibonacci terms.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB -2

1.AIM:***Write aprogramto perform linear search on a given input.***

PROGRAM:

```
deflinear_search(arr, key):
    foriinrange(len(arr)):
     ifarr[i]==key:
         return i
return -1


n=int(input("Enter no. of elements: ")) arr =

[]

print("Entertheelements:")

for i in range(n):
    arr.append(int(input()))

key=int(input("Enter element to search: "))

result=linear_search(arr, key)

if result != -1:
    print("Elementfound at index", result)
else:
    print("Elementnot found.")
```

OUTPUT:

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DS
A/linear search.py
Enter no. of elements: 7
Enter the elements:
23
46
67
78
98
96
36
Enter element to search: 98
Element found at index 4
>
```

EXPLANATION:

•The program implements the Linear Search algorithm.
•The user enters the number of elements. •The elements are stored in an array/list.
•The user enters the element to be searched.
•The linear_search() function checks the elements one by one. •Ifthe element matches the key, its index is returned.
•Ifthe element is not found, -1 is returned.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2.AIM

Write a Pythonprogramtoperform binary search on the given input.Verifybytakingsortedarrayand unsorted array as input

PROGRAM:

```
def binary_search(arr, key):
    low= 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low=mid+ 1
        else:
            high=mid- 1

    return -1


n=int(input("Enterno.ofelements: ")) arr =

[]

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

key=int(input("Enterelementtosearch: ")) result

= binary_search(arr, key)

if result!=-1:
    print("Elementfoundatindex",result)
else:
    print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/
Enter no. of elements: 6
Enter the elements:
23
56
89
90
56
34
Enter element to search: 90
Element not found.
```

EXPLANATION:

•The program implements Binary Search on a sorted array. •The user enters elements in sorted order.
•The algorithm uses low, high, and mid values.
•The middle element is calculated using (low + high) // 2.
•If the middle element equals the search key, its index is returned.
•If the key is greater than the middle element, the right half is searched. • If the key is smaller, the left half is searched.
•If the element is not found, -1 is returned.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid +1 or high = mid - 1.

Program:

```
# Binary Search of Unsorted Array
# CSE25105

arr = []

# Read number of elements
n=int(input("Enternoofelements:"))

# Read elements
for i in range(n):
    element=int(input("Entertheelement: "))
    arr.append(element)

# Check whether the list is sorted if
arr == sorted(arr):
    print("\nTheinputlistisalreadysorted.")
else:
    print("\nTheinputlistisnotsorted.")
    print("\nSorting the list...")
    arr.sort()

# Display sorted list
print("\nThe sorted list is:", arr)

# Read element to search
key=int(input("\nEnterelementtosearch: "))


# Binary Search Function def
binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2 if

        arr[mid] == key:
            return mid
```

```
        elifarr[mid] < key:
            low=mid + 1

        else:
            high=mid - 1

    return -1


#Performbinary search
result=binary_search(arr, key)

#Displayresult
if result!=-1:
    print("Element found at index:", result)
else:
    print("Element not found.")
```

Output:



Error Table:

Error	Rectification
Usedsortedmethod instead of whichneedtobe referred to anothervariable as it returns a sorted array	Used the arr.sort() method which directly sorts the art without using another variable

Description:

•The program performs Binary Search on user-provided elements.
•The input elements may initially be unsorted.
•The program checks whether the list is already sorted. •If the list is not sorted, arr.sort() is used.
•The sorted list is displayed.
•The user enters the element to be searched.
•Binary Search is then performed on the sorted list. •The middle element is compared with the search key. •If the element is found, its index is displayed. •Otherwise, "Element not found" is displayed.

LAB – 3

1.AIM : Write apython program to implement bubblesortwhere the input elements are takenfrom theuser

PROGRAM:

```
# Bubble Sort
def bubble_sort(arr):
    n = len(arr)
  for i in range(n - 1):
   for j in range(n - i - 1):
  if arr[j] > arr[j + 1]:
    arr[j],arr[j+1]=arr[j + 1], arr[j]

    return arr

    #Takinginput
    n=int(input("Enternumberof elements: "))

    arr = []

    print("Enter elements:")

    fori inrange(n):

    a rr. ap pen d(i nt(i np ut( )) )

    bub ble _so rt( ar r)

    print("Sorted array:")

    for elementin arr:

    print(element, end=" ")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/BUBBLE SORT.py ==========
Enter number of elements: 6
Enter elements:
34
56
90
87
56
23
Sorted array:
23 34 56 56 87 90
```

Explanation

Description:

The program implements the Bubble Sort algorithm.
•The user enters the number of elements.
•The elements are stored in a list.
•Adjacent elements are compared with each other.
•If the first element is greater than the second element, they are swapped.
•This process is repeated for multiple passes.
•Larger elements gradually move toward the end of the list.
•The bubble_sort() function performs the sorting.

Error table:

S.no	Error	Correction
1.	Used wrong loop range	Use for j in range(n-i-1):
2.	Swap condition is wrong	Use if arr[j] > arr[j+1]:

2.AIM:

Writeaprogramtoimplement insertion sort where the input elements are taken from the user

PROGR AM :

```
# InsertionSort

def insertion_sort(arr):
 n = len(arr)

 for i in range(1, n):

 key = arr[i]
 j = i - 1
 whilej>=0andarr[j] > key:

 arr[j + 1] = arr[j]
 j = j - 1

arr[j + 1] = key

returnarr

# Taking input
n=int(input("Enternumber of elements: "))


arr = []

print("Enter elements:")

for i inrange(n):
```

```
    arr.append(int(input()))

insertion_sort(arr)

print("Sorted array:")

forelement in arr:
    print(element, end=" ")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/INSERTION.PY ==========
Enter number of elements: 6
Enter elements:
34
56
08
20
05
89
Sorted array:
5 8 20 34 56 89
```

Explanation:

•The program implements the Insertion Sort algorithm.
•The user enters the number of elements.
•The elements are stored in a list.
•The first element is considered as the initially sorted portion.
•Each subsequent element is stored in a variable called key.
•The key is compared with the elements before it.
•Elements greater than the key are shifted one position to the right.
•The key is inserted into its correct position.
•This process continues until all elements are sorted.

Error table:

S.no	Error	Correction
1.	key is not stored properly	Use key = arr[i]
2.	arr[j+1] = key placed inside while loop	Place it after the while loop

3.AIM :

Writeapythonprogramto implement selection sort where the input elements are taken from the user.

PROGR AM :

```
# SelectionSort

defselection_sort(arr):
    n = len(arr)

    for i inrange(n- 1):
        min_index = i

        forjinrange(i+1,n):
            ifarr[j]<arr[min_index]:
                min_index = j

        arr[i],arr[min_index] = arr[min_index], arr[i]

    returnarr


# Taking input
n=int(input("Enternumber of elements: "))


arr = []

print("Enter elements:")

for i inrange(n):
    arr.append(int(input()))

selection_sort(arr)

print("Sorted array:")

for element inarr:
    print(element,end="")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/SELECTION SORT.py ==========
Enter number of elements: 7
Enter elements:
23
89
67
47
36
90
45
Sorted array:
23 36 45 47 67 89 90
```

Explanation:

•The program implements the Selection Sort algorithm.
•The user enters the number of elements.
•The elements are stored in a list.
•The algorithm starts from the first unsorted element.
•The smallest element in the remaining list is identified.
•Its index is stored in min_index.
•The smallest element is swapped with the current element.
•The process is repeated for all positions in the list.

Error table:

S.no	Error	Correction
1.	min_index is not initialized	Use min_index = i
2.	Swap is placed inside inner loop	Place swap after the inner loop

LAB-4

4a)

Aim:

Writeapythonprogramtoimplementmerge sort where the input elements are taken from the user.

Program

```
def merge_sort(arr):
if len(arr) <= 1:
return arr
mid = len(arr) // 2 left =
arr[:mid] right = arr[mid:]
merge_sort(left)
merge_sort(right)


 i = j = k = 0


 whilei<len(left)andj< len(right):
    if left[i] <= right[j]:
       arr[k] =
       left[i] i+=1
    else:
       arr[k] = right[j]
       j+=1
    k+=1


 while i < len(left):
    arr[k] = left[i] i
    += 1
```

```
        k += 1

    whilej<len(right):
        arr[k]=right[j]
        j+=1
        k+=1

n=int(input("Enter number of elements: ")) arr =
[]

print("Enter elements:")
foriin range(n):
    arr.append(int(input()))

merge_sort(arr)

print("Sorted Array:")
forelement in arr:
    print(element, end=" ")
```

Output:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/merge sort.py ==========
Enter number of elements: 5
Enter elements:
45
56
67
90
23
Sorted Array:
23 45 56 67 90
```

Error table

error	rectification
TypeError:object of type ‘NoneType’ has no len()	Taken return arr at end of merge_sort

Explanation:

•The program implements the Merge Sort algorithm.

•The user enters the size of the array.

•The elements are taken from the user and stored in a list.

•The merge_sort() function recursively divides the array into two halves. •The array is divided until each sub-array contains one or zero elements. •The left and right portions are then sorted recursively.

•The sorted left and right arrays are compared element by element.

•The smaller element is placed into the original array.

•Remaining elements from either portion are then copied into the array. •Finally, the completely sorted array is displayed.

•Merge Sort follows the divide-and-conquer approach.

4b)

Aim:

Write a pythonprogramtoimplement quick sort where the input elementsaretakenfromtheuser. def quicksort(a, low, high):

```
    iflow<high:
        i=low+1 j
        =high pivot
        = low

        while i <= j:
            whilei<=highanda[i]<= a[pivot]:
                i+=1

            while a[j] > a[pivot]:
                j -= 1

            if i < j:
                a[i], a[j] = a[j], a[i]

        a[j],a[pivot]=a[pivot],a[j]

        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)


a=list(map(int,input("Enternumbers to sort: ").split())) n = len(a)


quicksort(a, 0, n - 1)

print("Sorted array:")
```

OUTPUT:



Error table

Error	Rectification
i<len(a)checks the complete array Use i <= high and maintain correct instead ofcurrent subarray	partition boundaries

Description:

• The program implements the Quick Sort algorithm.

• The user enters the size of the array.

• The elements are taken from the user and stored in a list.

• The quick_sort() function is used to sort the array recursively.

• The first element of the current portion is selected as the pivot.

• Two pointers, i and j, are used to compare elements with the pivot.

• i moves from the left side and searches for an element greater than or equal to the pivot.

• j moves from the right side and searches for an element smaller than or equal to the pivot.

•If i < j, the two elements are swapped.

•After the partitioning process, the pivot is placed in its correct position. •The array is then divided into left and right portions.

•Quick Sort is recursively applied to both portions.

•Finally, the sorted array is printed.

Week-5

5a)

Aim: ➢ WriteaPythonprogramtoimplement a Singly Linked List (SLL) with the following operations:

1. CreateLL

2. InsertatBeginning 3.

InsertatEnd

4. InsertatIndex 5.

DeletebyValue

6. DeletePriorNode

7. DeleteLastNode

8. CountNumberofNode 9.

Display/Traverse

10. Exit Program:

Program:

```
#Singly Linked List
#CSE25105

print("Implementation of SLL")



#NodeClass
classNode:
    def __init__(self, data):
        self.data = data
        self.next = None



#SinglyLinked List Class
classSingleLinkedList:
    def __init__(self):
```

```
    self.head = None

# Creation
def create(self):
    n=int(input("Enternoofelements: "))

    for i in range(n):
        data=int(input("Enterthe value: ")) new
        = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new

# Insert at Beginning
def insert_begin(self, data):
    new = Node(data)
    new.next = self.head
    self.head = new

# Insert at End
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
```

```
    else:
        temp=self.head

        whiletemp.next:
            temp=temp.next

        temp.next=new

#InsertatSpecificIndex
definsert_index(self, index, data):

    if index == 0:
        self.insert_begin(data)
        return

    elif index < 0:
        print("Invalidindex")
        return

    else:
        new=Node(data)
        temp=self.head

        foriinrange(index - 1):
            iftempisNone or temp.next is None:
                print("Invalid index")
                return

            temp=temp.next

        new.next=temp.next
        temp.next=new
```

```
# Delete Specific Value
def delete(self, data):

    if self.head is None:
        print("No data")
        return

    temp = self.head

    #Iffirstnodecontainsthe value if
    temp.data == data:
        self.head=temp.next
        print("Valuedeleted")
        return

    # Search for the value
    whiletemp.nextandtemp.next.data != data:
        temp = temp.next

    if temp.next is None:
        print("Valuenotpresent")
    else:
        temp.next=temp.next.next
        print("Valuedeleted")

# Delete Beginning
def delete_begin(self):

    if self.head is None:
        print("Nodatatodelete")
    else:
```

```
        temp = self.head
        self.head = temp.next print("Deleted
        value =", temp.data)

#DeleteEnd
defdelete_end(self):

    ifself.head is None:
        print("No data to delete")

    elifself.head.next is None:
        print("Deleted value =", self.head.data)
        self.head = None

    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted value =", temp.next.data)
        temp.next = None

#CountNodes
defcount(self):

    ifself.head is None:
        print("No data to count")
    else:
        count = 0
        temp = self.head
```

```
            while temp:
                count += 1
                temp=temp.next

            print(f"Numberofnodes = {count}")

    # Display / Traverse
    def display(self):

        if self.head is None:
            print("No data")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" --> ")
                temp=temp.next

            print("None")



# Main Program
SLL = SinglyLinkedList()

while True:

    print("\n----- SINGLY LINKED LIST -----
    ") print("1. Create SLL")
    print("2. Insert at beginning")
    print("3. Insert at ending")
    print("4. Insert at index")
    print("5. Delete by value")
```

```
print("6. Delete first node")
print("7. Delete last node")
print("8. Count no. of nodes")
print("9. Display / Traverse")
print("10. Exit")

choice = int(input("Enter your choice: "))

ifchoice == 1:
    SLL.create() print("SLL
    is created")

elifchoice == 2:
    data = int(input("Enter data to insert: "))
    SLL.insert_begin(data)
    print("Data is inserted at beginning")

elifchoice == 3:
    data = int(input("Enter data to insert: "))
    SLL.insert_end(data)
    print("Data is inserted at ending")

elifchoice == 4:
    data = int(input("Enter data to insert: "))
    index = int(input("Enter the index: "))
    SLL.insert_index(index, data)

elifchoice == 5:
    data = int(input("Enter data to delete: "))
    SLL.delete(data)

elifchoice == 6:
```

```
            SLL.delete_begin()

        elifchoice == 7:
            SLL.delete_end()

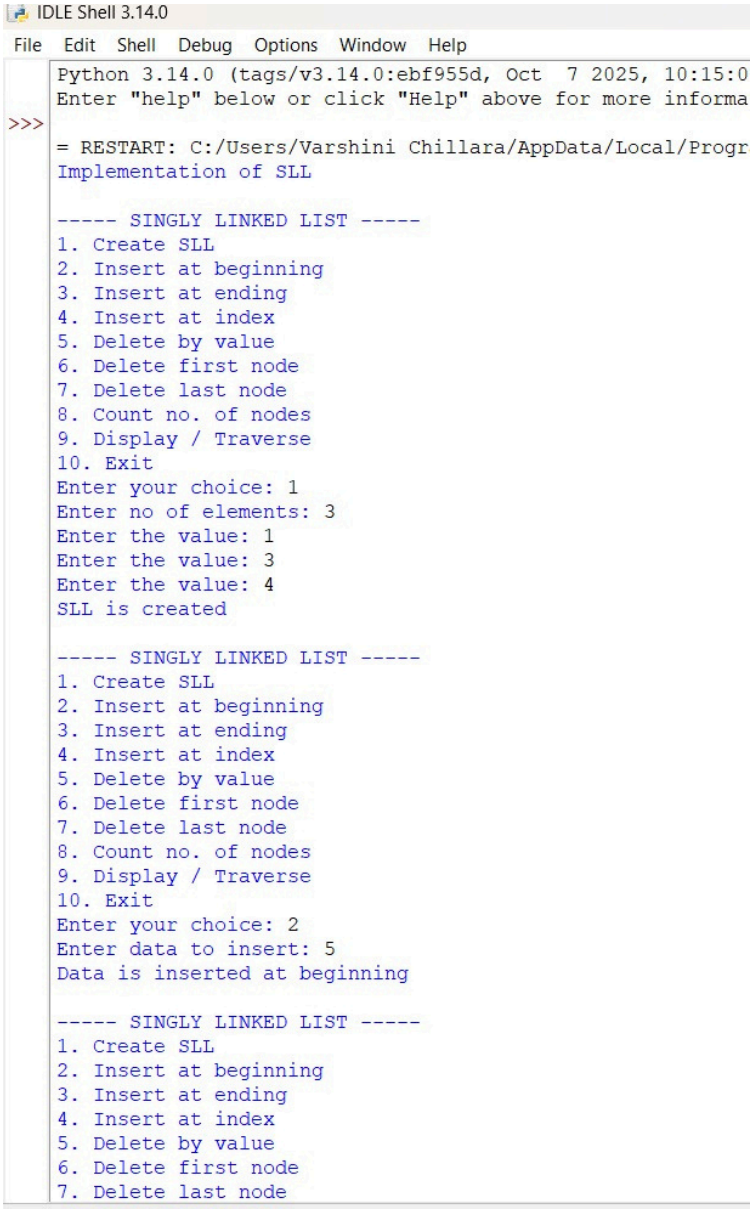
        elifchoice == 8:
            SLL.count()

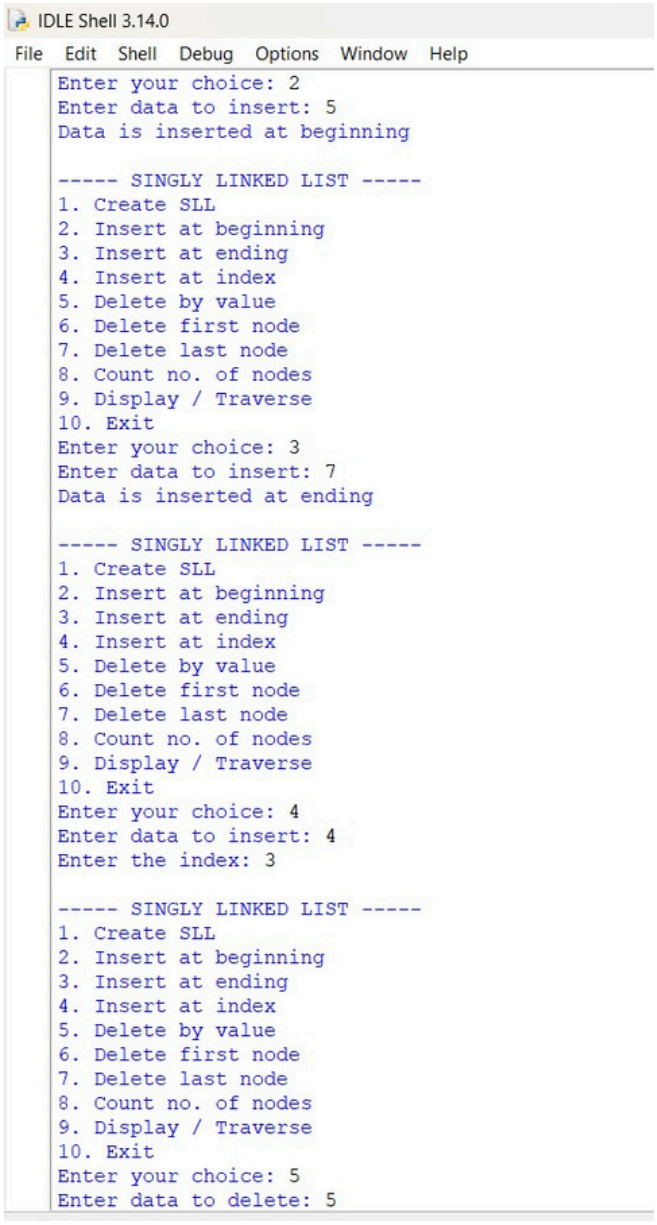
        elifchoice == 9:
            SLL.display()

        elifchoice == 10:
            print("Exiting the program...")
            break

        else:
            print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:0
Enter "help" below or click "Help" above for more informa
>>>
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Progr
Implementation of SLL
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter no of elements: 3
Enter the value: 1
Enter the value: 3
Enter the value: 4
SLL is created
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data to insert: 5
Data is inserted at beginning
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Enter your choice: 2
Enter data to insert: 5
Data is inserted at beginning
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data to insert: 7
Data is inserted at ending
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter data to insert: 4
Enter the index: 3
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter data to delete: 5

```
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
Deleted value = 1

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Deleted value = 7

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes = 3
```

```
----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
3 --> 4 --> 4 --> None

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Exiting the program...
```

Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

- The program implements a **Singly Linked List** using Python classes.
- The `Node` class is used to create individual nodes.
- Each node contains two parts: `data` and `next`.
- `data` stores the value of the node.
- `next` stores the reference to the next node.
- The `SingleLinkedList` class manages the linked list.
- The `head` variable stores the reference to the first node.
- The program provides an option to **create** a linked list.
- It supports **insertion at the beginning** of the list.
- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** all the nodes.
- A menu-driven approach is used to allow the user to select different operations.
- The program continues running until the user selects the **Exit** option.

5b)

Aim:

➢ Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
#Doubly Linked List
#CSE25106


print("Implementation of DLL")



#Node Class
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
```

```
        self.next = None



#DoublyLinkedListClass
classDoubleLinkedList:
    def __init__(self):
        self.head = None

    # Creation
    def create(self):
        n=int(input("Enter no of elements: "))

        for i in range(n):
            data=int(input("Enter the value: "))
            new=Node(data)

            ifself.headisNone:
                self.head=new
            else:
                temp=self.head

                whiletemp.next:
                    temp=temp.next

                temp.next=new
                new.prev=temp
```

```
#InsertatBeginning
definsert_begin(self, data):
    new=Node(data)

    ifself.headisNone:
        self.head=new
    else:
        new.next=self.head
        self.head.prev = new
        self.head=new

# Insert at End
definsert_end(self, data):
    new=Node(data)

    ifself.headisNone:
        self.head=new
    else:
        temp=self.head

        whiletemp.next:
            temp=temp.next

        temp.next=new
        new.prev=temp
```

```
#InsertatSpecificIndex
definsert_index(self,index, data):

    if index < 0:
        print("Invalidindex")
        return

    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalidindex")
        return

    temp = self.head

    foriinrange(index-1):
        iftemp.nextisNone:
            print("Invalidindex")
            return

        temp = temp.next

    new = Node(data)
```

```
    new.next = temp.next
    new.prev = temp

    iftemp.next is not None:
        temp.next.prev = new

    temp.next = new

#Delete Specific Value
defdelete(self, data):

    ifself.head is None:
        print("No data")
        return

    temp = self.head

    #Search for the value
    while temp and temp.data != data:
        temp = temp.next

    iftemp is None:
        print("Value not present")
        return
```

```
    #Ifdeleting the first node
    iftemp== self.head:
        self.head = temp.next

        ifself.head is not None:
            self.head.prev = None

    else:
        #Connect previous node to next node
        temp.prev.next = temp.next

        #Connect next node to previous node
        iftemp.next is not None:
            temp.next.prev = temp.prev

    print("Value deleted")

#DeleteBeginning
defdelete_begin(self):

    ifself.head is None:
        print("No data to delete")
        return

    temp=self.head
    self.head= temp.next
```

```
    ifself.head is not None:
        self.head.prev = None

    print("Deleted value =", temp.data)

#Delete End
defdelete_end(self):

    ifself.head is None:
        print("No data to delete")
        return

    temp = self.head

    #Only one node
    iftemp.next is None:
        print("Deleted value =", temp.data)
        self.head = None
        return

    #Move to last node
    while temp.next:
        temp = temp.next

    print("Deleted value =", temp.data)
```

```
    temp.prev.next = None

#Count Nodes
defcount(self):

    ifself.head is None:
        print("No data to count")
        return

    count = 0
    temp = self.head

    while temp:
        count += 1
        temp = temp.next

    print("Number of nodes =", count)

#Display / Traverse
defdisplay(self):

    ifself.head is None:
        print("No data")
        return
```

```
        temp = self.head

        while temp:
            print(temp.data, end=" <--> ")
            temp = temp.next

        print("None")



#MainProgram
DLL=DoubleLinkedList()

while True:

    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create DLL")
    print("2. Insert at beginning")
    print("3. Insert at ending")
    print("4. Insert at index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))

ifchoice == 1:
    DLL.create()
    print("DLL is created")

elifchoice == 2:
    data = int(input("Enter data to insert: "))
    DLL.insert_begin(data)
    print("Data is inserted at beginning")

elifchoice == 3:
    data = int(input("Enter data to insert: "))
    DLL.insert_end(data)
    print("Data is inserted at ending")

elifchoice == 4:
    data = int(input("Enter data to insert: "))
    index = int(input("Enter the index: "))
    DLL.insert_index(index, data)

elifchoice == 5:
    data = int(input("Enter data to delete: "))
    DLL.delete(data)

elifchoice == 6:
```

```
        DLL.delete_begin()

    elifchoice == 7:
        DLL.delete_end()

    elifchoice == 8:
        DLL.count()

    elifchoice == 9:
        DLL.display()

    elifchoice == 10:
        print("Exiting the program")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

•Theprogram implements a Doubly Linked List using Python classes.
•TheNode class is used to create individual nodes.
•Eachnode contains three parts: data, prev, and next.
•datastores the value of the node.
•prevstores the reference to the previous node.
•nextstores the reference to the next node.
•TheDoubleLinkedList class is used to manage the linked list.
•Thehead variable stores the reference to the first node. •Theprogram provides an option to create a doubly linked list. •Itsupports insertion at the beginning of the list.
•Itsupports insertion at the end of the list.
•Itsupports insertion at a specific index.
•Itsupports deletion of a specific value.
•Itsupports deletion of the first node. •Itsupports deletion of the last node.
•Itprovides a function to count the number of nodes. 36
•Itprovides a function to display/traverse the list.
•During insertion and deletion, both prev and next links are updated.
•Thelist is displayed using <--> to show the connection between nodes. •Amenu-driven approach is used to perform different DLL operations.
•Theprogram continues until the user selects the Exit option.

5c)

Aim:

➢WriteaPythonprogramtoimplement a Circular Linked List (CLL) with the following operations: 1. Create LL

2.InsertatBeginning
3.InsertatEnd 4.InsertatIndex
5. Delete by Value
6.DeletePriorNode
7. Delete Last Node
8. Count Number of Nodes

9. Display/Traverse
10. Exit

Program:

```
# Circular Linked List

# CSE25105

print("Implementation of CLL")

# Node Class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Circular Linked List Class
class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Creation
    def create(self):
        n=int(input("Enternoofelements: ")) for i in

        range(n):
            data=int(input("Enterthe value: ")) new =
            Node(data)

            if self.head is None:
                self.head = new
                new.next = self.head
            else:
                temp = self.head

                whiletemp.next!=self.head:
                    temp = temp.next

                temp.next = new
                new.next = self.head

    # Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
```

```
        temp = self.head

        whiletemp.next!=self.head:
            temp = temp.next

        new.next = self.head
        self.head = new
        temp.next = self.head

# Insert at End
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        whiletemp.next!=self.head:
            temp = temp.next

        temp.next = new
        new.next = self.head

# Insert at Specific Index
definsert_index(self,index,data):

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid index")
        return

    temp = self.head

    for i in range(index - 1):
        iftemp.next==self.head:
            print("Invalid index")
            return

        temp = temp.next

    new = Node(data)
    new.next = temp.next
    temp.next = new

# Delete Specific Value
def delete(self, data):

    if self.head is None:
        print("No data")
        return

    #Ifheadcontainsthevalue if
    self.head.data == data:

        # Only one node
        ifself.head.next==self.head:
```

```
            self.head = None
            print("Value deleted")
            return

        # More than one node
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = self.head.next
        self.head = self.head.next

        print("Value deleted") return


    # Search for the value
    temp = self.head

    whiletemp.next!=self.headandtemp.next.data != data:
        temp = temp.next

    if temp.next == self.head:
        print("Value not present")
    else:
        temp.next = temp.next.next
        print("Value deleted")

# Delete Beginning
def delete_begin(self):

    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deletedvalue=",self.head.data)
        self.head = None
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    print("Deletedvalue=",self.head.data)

    temp.next = self.head.next
    self.head = self.head.next

# Delete End
def delete_end(self):

    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deletedvalue=",self.head.data)
        self.head = None
        return

    temp = self.head
```

```
        whiletemp.next.next!=self.head:
            temp = temp.next

        print("Deletedvalue=",temp.next.data)

        temp.next = self.head

    # Count Nodes
    def count(self):

        if self.head is None:
            print("No data to count")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Numberofnodes=",count)

    # Display / Traverse
    def display(self):

        if self.head is None:
            print("No data")
            return

        temp = self.head

        while True:
            print(temp.data,end="-->") temp
            = temp.next

            if temp == self.head:
                break

        print("HEAD")


# Main Program
CLL = CircularLinkedList()

while True:

    print("\n-----CIRCULARLINKEDLIST -----")
    print("1. Create CLL")
    print("2. Insert at beginning")
    print("3. Insert at ending")
    print("4. Insert at index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice=int(input("Enteryourchoice: "))
```

```
ifchoice == 1:
    CLL.create() print("CLL
    is created")

elifchoice == 2:
    data = int(input("Enter data to insert: "))
    CLL.insert_begin(data)
    print("Data is inserted at beginning")

elifchoice == 3:
    data = int(input("Enter data to insert: "))
    CLL.insert_end(data)
    print("Data is inserted at ending")

elifchoice == 4:
    data = int(input("Enter data to insert: ")) index
    = int(input("Enter the index: "))
    CLL.insert_index(index, data)

elifchoice == 5:
    data = int(input("Enter data to delete: "))
    CLL.delete(data)

elifchoice == 6:
    CLL.delete_begin()

elifchoice == 7:
    CLL.delete_end()

elifchoice == 8:
    CLL.count()

elifchoice == 9:
    CLL.display()

elifchoice == 10:
    print("Exiting the program")
    break

else:
    print("Invalid choice")
```

Output:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/c:
Implementation of CLL

----- CIRCULAR LINKED LIST -----
1. Create CLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter no of elements: 4
Enter the value: 2
Enter the value: 4
Enter the value: 6
Enter the value: 8
CLL is created

----- CIRCULAR LINKED LIST -----
1. Create CLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice:
```

Error Table:

Error	Rectification
No major error is present in the given program.	No major rectification is required.

Description:

- The program implements a **Circular Linked List** using Python classes.
- The `Node` class is used to create individual nodes.
- Each node contains two parts: `data` and `next`.
- `data` stores the value of the node.
- `next` stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The `CircularLinkedList` class is used to manage the linked list.
- The `head` variable stores the reference to the first node.
- The program provides an option to **create** a circular linked list.
- It supports **insertion at the beginning** of the list.

- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** the circular linked list.
- During traversal, the program stops when it reaches the `head` again.
- The list is displayed using `-->` and ends with **HEAD**, showing its circular nature.
- The program handles the special case where the list contains only one node.
- A **menu-driven approach** is used to perform different CLL operations.
- The program continues until the user selects the **Exit** option.

Week-6

6a)

Aim:

➢ WriteaPythonprogramtoimplement stack operations such as Push, Pop, Peek, and Display using an Array.

Pro gram:

```
# Stack using Array
# CSE25105


class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    # Push operation
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item,"pushedintothestack")

    # Pop operation
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item,"poppedfromthestack")

    # Peek operation
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Topelement:",self.stack[self.top])

    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Theelementsofthestack:") for i in

            range(self.top, -1, -1):
                print(self.stack[i])


# Main Program

size=int(input("Enterthesizeofthestack: ")) s =

Stack(size)

while True:
```

```
print("\n-----STACK MENU -----") print("1.
Push")
print("2. Pop")
print("3. Peek")
print("4. Display")
print("5. Exit")

choice=int(input("Enter your choice: ")) if
choice == 1:

    item=int(input("Enter the element to push: "))
    s.push(item)

elif choice == 2:
    s.po p()

elif choice == 3:
    s.peek()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Exitingprogram")
    break

else:
    print("Invalidchoice")
```

OUTPUT:



Error Table:

Error	Rectification
In peek, self.peek is taken in 'if' does not give correct answer for stack underflow.	Self.top is taken in 'if' to give correct output.

Description:

- The `Stack` class implements a stack using a Python list.
- `size` stores the maximum size of the stack.
- `stack` stores the stack elements.
- `top` keeps track of the top position. Initially, `top = -1`, meaning the stack is empty.
- **Push** adds an element to the top of the stack.
- **Pop** removes the top element from the stack.

- **Peek** displays the top element without removing it.
- **Display** prints all elements from top to bottom.
- **Stack Overflow** occurs when trying to push into a full stack.
- **Stack Underflow** occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b)

Aim:

➢ WriteaPythonprogramtoimplement stack operations such as Push, Pop, Peek, and Display using an Linked List.

Pro gram:

```
# Stack using Linked List #
CSE25105


# Node Class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Stack Class
class Stack:
        def __init__(self):
            self.top = None

    # Push Operation
    def push(self, item):
    new = Node(item)
    new.next = self.top
    self.top = new

    print(item,"pushedintothestack")

    # Pop Operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            self.top = self.top.next

            print(temp.data,"poppedfrom the stack")

    # Peek Operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Topelement:",self.top.data)

    # Display Operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Theelementsofthestack:")
```

```
        temp = self.top

        whiletempisnot None:
            print(temp.data)
            temp=temp.next


# Main Program

s = Stack() while

True:

    print("\n-----STACKMENU -----") print("1.
    Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice=int(input("Enter your choice: ")) if
    choice == 1:

        item=int(input("Enter the element to push: "))
        s.push(item)

    elif choice == 2:
        s.po p()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Exitingprogram")
        break

    else:
        print("Invalidchoice")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 4
4 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
4 popped from the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

Error Table:

Error	Rectification
No major error in the linked-list implementation.	Not required.

Description:

- The stack is implemented using a **singly linked list**.
- Each node contains `data` and a `next` pointer.
- `top` points to the top node of the stack.
- **Push** inserts a new node at the beginning of the linked list.
- **Pop** removes the node pointed to by `top`.
- **Peek** displays the data of the top node without removing it.
- **Display** traverses the linked list from `top` to the last node.
- If `top` is `None`, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.

Lab-7

1.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array

Program:

```
#QueueImplementation using Linked List
#Operations: Enqueue, Dequeue, Peek, Display

classNode:
    def__init__(self, data):
        self.data = data
        self.next = None


classQueue:
    def__init__(self):
        self.front = None
        self.rear = None

    #Enqueue operation
    defenqueue(self, data):
        new_node = Node(data)

        ifself.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into queue.")

    #Dequeue operation
    defdequeue(self):
        ifself.front is None:
            print("Queue Underflow! Queue is empty.")
            return

        value = self.front.data
        self.front = self.front.next
```

```
        if self.front is None:
            self.rear = None

        print(value, "deleted from queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Frontelement:",self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty.")
            return

        current = self.front

        print("Queue elements:", end=" ")

        while current is not None:
            print(current.data, end=" ")
            current = current.next

        print()


# Create Queue object
queue = Queue()

# Menu-driven program
while True:
    print("\n-----QUEUEUSINGLINKEDLIST-----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value=int(input("Enterelementtoenqueue: "))
        queue.enqueue(value)

    elif choice == 2:
        queue.dequeue()

      elif choice == 3:
          queue.peek()

      elif choice == 4:
        queue.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalidchoice!Pleasetryagain.")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/queuelinke

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 24
24 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 45
45 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
24 deleted from queue.
```

```
----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
24 deleted from queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 34

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 34 45

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

Aqueue is a linear data structure that follows the FIFO (First In, First Out) principle. In this program, Python's list is used as an array to store queue elements.

Error table:

S.No	Error	Rectification
1	Using `queue.pop()` instead of `queue.pop(0)` removes the last element instead of the front element.	Use `queue.pop(0)` for dequeue operation.
2	Not checking the queue size before insertion can cause the queue to exceed its maximum size.	Check `if len(queue) == MAX_SIZE` before enqueue.
3	Accessing `queue[0]` when the queue is empty causes an `IndexError`.	Check `if len(queue) == 0` before performing Peek.

2.Aim:

WriteaPythonprogramtoimplement a queue and its operations such as Enqueue, Dequeue, Peek, Displaywithamenu-drivenapproach. Implement using linked-list data structures. Program:

```
#QueueImplementationusingLinked List

#Operations:Enqueue,Dequeue,Peek, Display

class Node:

    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data) if

        self.rear is None:
            self.front=self.rear=new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(data,"insertedintoqueue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("QueueUnderflow!Queue is empty.") return


        value = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        print(value,"deletedfromqueue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Frontelement:",self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty.")
            return

        current = self.front

        print("Queueelements:",end=" ")
```

```
        whilecurrentisnot None:
            print(current.data, end=" ")
            current=current.next

        print()


# Create Queue
queue = Queue()

# Menu-driven program
while True:
    print("\n-----QUEUEUSING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice=int(input("Enter your choice: "))

    if choice == 1:
        value=int(input("Enter element to enqueue: "))
        queue.enqueue(value)

    elif choice == 2:
        queue.dequeue()

      elif choice == 3:
          queue.peek()

      elif choice == 4:
        queue.display()

    elif choice == 5:
        print("Programterminated.")
        break

    else:
        print("Invalidchoice! Please try again.")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/QUEUE.PY

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 78
78 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 40
40 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from queue.
```

```
----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 78

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 78 40

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

Alinked-list queue consists of dynamically created nodes. Each node contains two parts:

Two pointers are maintained:

Front: Points to the first node.

Rear: Points to the last node.

Error table:

S.No	Error	Rectification
1	After deleting the last node, `rear` may still point to the deleted node.	If `front` becomes `None`, set `self.rear = None`.
2	Displaying the linked list without moving to the next node can create an infinite loop.	Use `current = current.next` inside the display loop.

3.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek,

Display with a menu-driven approach. Implement using Circular Queue Arrays structures.

Program:

```
# Circular QueueImplementationusing Array
# Operations:Enqueue,Dequeue,Peek, Display

MAX_SIZE = 5

queue = [None] * MAX_SIZE
front = -1
rear = -1

while True:

    print("\n-----CIRCULARQUEUE-----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice=int(input("Enteryourchoice: "))

    # Enqueue
    if choice == 1:

        # Check whether queue is full
        if(rear+1)%MAX_SIZE==front:
            print("QueueOverflow!Circular Queue is full.")

        else:
            value=int(input("Enterelement to enqueue: "))

            # First element
            if front == -1:
                front = 0
                rear = 0

            else:
                rear=(rear+1)%MAX_SIZE

            queue[rear] = value

            print(value,"insertedintocircular queue.")

    # Dequeue
    elif choice == 2:

        if front == -1:
            print("QueueUnderflow!Circular Queue is empty.")

        else:
            value = queue[front]
            print(value,"deletedfromcircular queue.")
            #Onlyoneelementwaspresent
            if front == rear:


                front = -1
```

```
            rear = -1

        else:
            front=(front+1) % MAX_SIZE

# Peek
elif choice == 3:

    if front == -1:
        print("Queueisempty.")

    else:
        print("Frontelement:", queue[front])

# Display
elif choice == 4:

    if front == -1:
        print("Queueisempty.")

    else:
        print("CircularQueue elements:", end=" ") i =

        front

        while True:
            print(queue[i],end=" ") if i

            == rear:
                break

            i=(i+1)%MAX_SIZE

        print()

# Exit
elif choice == 5:
    print("Programterminated.")
    break

# Invalid choice
else:
    print("Invalidchoice!Please try again.")
```

OUTPUT:

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/CIRCULARQUEUE.PY ==

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 48
48 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 46
46 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from circular queue.
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 48

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 48 46

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

A circular queue is a queue in which the last position is connected back to the first position.

Unlike a normal linear queue, a circular queue can reuse empty positions created after dequeue operations.

Error table:

S.No	Error	Rectification
1	Using `rear + 1` without `% MAX_SIZE` can make the rear go outside the array.	Use `rear = (rear + 1) % MAX_SIZE`.
2	Not resetting both pointers after deleting the last element can leave the queue in an incorrect state.	When `front == rear`, set both `front = -1` and `rear = -1`.

4.Aim:

WriteaPythonprogramtoimplementaqueue and its operations such as Enqueue, Dequeue, Peek, Displaywithamenu-drivenapproach.Implement using Circular Queue linked-list data structures. Program:

```
# Circular Queue using Singly Linked List
#Operations:Enqueue,Dequeue,Peek,Display



class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        # Queue is empty
        if self.rear is None:
            self.rear = new_node
            self.rear.next = self.rear

        else:
            # New node is inserted after rear
            new_node.next = self.rear.next
            self.rear.next = new_node self.rear =
            new_node

        print(data,"insertedintocircularqueue.")

    # Dequeue operation
    def dequeue(self):
        if self.rear is None:
            print("QueueUnderflow!Queueisempty.") return


        # Front node
        front = self.rear.next

        # Only one node
        if self.rear == front:
            self.rear = None

        else:
            self.rear.next = front.next

        print(front.data,"deletedfromcircular queue.")

    # Peek operation
    def peek(self):
        if self.rear is None:
            print("Queue is empty.")

        else:
            print("Frontelement:",self.rear.next.data)

    # Display operation
```

```
    def display(self):
        if self.rear is None:
            print("Queueisempty.")
            return

        front = self.rear.next
        current = front


        print("CircularQueueelements:", end=" ") while

        True:
            print(current.data,end=" ")
            current = current.next

            if current == front:
                break

        print()


# Create circular queue
queue = CircularQueue()


# Menu-driven program
while True:

    print("\n-----CIRCULARQUEUE USING SINGLY LINKED LIST -----
    ") print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice=int(input("Enteryour choice: "))

    if choice == 1:
        value=int(input("Enterelement to enqueue: "))
        queue.enqueue(value)

    elif choice == 2:
        queue.dequeue()

      elif choice == 3:
          queue.peek()

      elif choice == 4:
        queue.display()

    elif choice == 5:
        print("Programterminated.")
        break

    else:
        print("Invalidchoice!Please try again.")
```

OUTPUT:

```
========================================= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/CIRCULARQUEUELINKED.PY ==

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 36
36 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 86
86 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 57
Invalid choice! Please try again.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 67
67 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
```

```
----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
36 deleted from circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 86

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 86 67

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

data → stores the element.

next → stores the address/reference of the next node.

This inserts the new node at the rear while maintaining the circular connection.

Error table:

S.No	Error	Rectification
1	Setting `rear.next = None` breaks the circular connection.	Set `rear.next = front` so the last node points to the first node.
2	Not setting `rear = None` when the only node is deleted leaves an invalid queue.	If `rear == front`, set `rear = None`.

LAB-8:

A.) Aim: write a python program to implement a binary tree with its operations using an array:

Program:

```
# BINARY TREE USING ARRAY
class BinarytreeArray:
  def __init__(self, size):
    self.tree = [None] * size
    self.size = size
  # Insert root
  def set_root(self, data):
    self.tree[0] = data
  # Set left child
  def set_left(self, parent_index, data):
    child_index = 2 * parent_index + 1
    if child_index < self.size:
      self.tree[child_index] = data
    else:
      print("Index out of range")
  # Set right child
  def set_right(self, parent_index, data):
    child_index = 2 * parent_index + 2
    if child_index < self.size:
      self.tree[child_index] = data
    else:
      print("Index out of range")
  # Preorder Traversal
  def preorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
      return


    print(self.tree[index], end=" ")
    self.preorder(2 * index + 1)
    self.preorder(2 * index + 2)
  # Inorder Traversal
  def inorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
      return
```

```
    self.inorder(2 * index + 1)
    print(self.tree[index], end=" ")
    self.inorder(2 * index + 2)
# Postorder Traversal
def postorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
        return
    self.postorder(2 * index + 1)
    self.postorder(2 * index + 2)
    print(self.tree[index], end=" ")
# Level-order Traversal
def level_order(self):
    for value in self.tree:
        if value is not None:
            print(value, end=" ")
# Display Array
def display(self):
    print("\nArray Representation")
    for i in range(self.size):
        print(f"Index {i} : {self.tree[i]}")

# MAIN PROGRAM

tree = BinarytreeArray(7)
# Create the tree
tree.set_root('A')
# Children of A
tree.set_left(0, 'B')
tree.set_right(0, 'C')
# Children of B
tree.set_left(1, 'D')
tree.set_right(1, 'E')
# Right child of C
tree.set_right(2, 'F')

# Display tree
tree.display()
```

```
# Traversals
print("\nPreorder: ", end="")
tree.preorder()
print("\nInorder: ", end="")
tree.inorder()
print("\nPostorder: ", end="")
tree.postorder()
```

OUTPUT

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/binary
treearray.py

Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

ERROR TABLE

S.No	Type of Error	Description
1	Syntax Error	Incorrect syntax in Python statements.
2	Indentation Error	Improper indentation inside functions and loops.
3	Index Error	Accessing an array position outside its valid range.

EXPLANATION

A binary tree is a non-linear data structure in which each node has at most two children: left child and right .

B.) **Aim:** write a python program to implement a binary tree with its operations using Linked list:

Program:

```
# BINARY TREE USING LINKED LIST
class Node:
  def __init__(self, data):
    self.data = data
    self.left = None
    self.right = None
# BINARY TREE CLASS
class BinaryTree:
  def __init__(self):
    self.root = None
  # Create the binary tree
  def create_tree(self):
    self.root = Node('A')
    # create root
    # Left and right children of A
    self.root.left = Node('B')
    self.root.right = Node('C')
    # Children of B
    self.root.left.left = Node('D')
    self.root.left.right = Node('E')
    # Right child of C
    self.root.right.right = Node('F')
  # Preorder Traversal
  def preorder(self, node):
    if node is None:
      return

    print(node.data, end="")
```

```
    self.preorder(node.left)
    self.preorder(node.right)
  # Inorder Traversal
  def inorder(self, node):
    if node is None:
      return
    self.inorder(node.left)
    print(node.data, end="")
    self.inorder(node.right)
  # Postorder Traversal
  def postorder(self, node):
    if node is None:
      return
    self.postorder(node.left)
    self.postorder(node.right)
    print(node.data, end=" ")
  # Level-order Traversal
  def level_order(self):
    if self.root is None:
      return
    queue = [self.root]
    while queue:
      node = queue.pop(0)
      # Remove first node
      print(node.data, end=" ")
      # Add left child
      if node.left is not None:
        queue.append(node.left)

      # Add right child
```

```
            if node.right is not None:
                queue.append(node.right)

# MAIN PROGRAM
tree = BinaryTree()
# Create the binary tree
tree.create_tree()
# Traversals
print("Preorder: ", end="")
tree.preorder(tree.root)
print("\nInorder: ", end="")
tree.inorder(tree.root)
print("\nPostorder: ", end="")
tree.postorder(tree.root)
print("\nLevel-order: ", end="")
tree.level_order()
```

OUTPUT

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/BINARY
TREE LINKEDLIST.PY
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

ERROR TABLE

S.No	Type of Error	Description
1	Syntax Error	Incorrect syntax while defining the Node class or functions.
2	Indentation Error	Improper indentation inside class methods and functions.
3	Attribute Error	Accessing left or right attributes of a node incorrectly.
4	Type Error	Assigning an invalid data type to a node.

EXPLANATION

A binary tree is a non-linear data structure in which each node contains data and two links (left and right).
In linked list representation, each node is created dynamically using a class.
The left pointer connects to the left child, and the right pointer connects to the right child.
The first node is called the root node, which is the starting point of the binary tree.
This representation allows dynamic memory allocation and makes insertion and deletion easier.

LAB-9:

A.) **Aim:** write a python program to implement a binary search tree with its operations and traversals:

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end="")
    inorder(root.right)


def preorder(root):
    if root is None:
```

```
        return
    print(root.data, end="")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end="")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
```

```
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
```

```
    return max(left_height, right_height) + 1

def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        #case 1 node has no children
        if root.left is None and root.right is None:
            root = None
        #case 2 node has onle right child
        elif root.left is None:
            root = root.right
        #case 3 node only left child
        elif root.right is None :
            root = root.left
        #case 4 node has 2 children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
```

```
    if level == 0:
        print(root.data, end="")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)

def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)

# MAIN PROGRAM

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")
```

```
choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
    print("Element inserted successfully.")
elif choice == 2:
    print("Preorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        preorder(root)
        print()
elif choice == 3:
    print("Inorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        inorder(root)
        print()
elif choice == 4:
    print("Postorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        postorder(root)
        print()
elif choice == 5:
    node = find_min(root)
    if node is None:
```

```
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
```

```
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
Enter your choice: 1
Enter the data to be inserted: 67
Element inserted successfully.
14. Exit
Enter your choice: 1
Enter the data to be inserted: 34
Element inserted successfully.

14. Exit
Enter your choice: 1
Enter the data to be inserted: 56
Element inserted successfully.

Enter your choice: 1
Enter the data to be inserted: 45
Element inserted successfully.
14. Exit
Enter your choice: 11
Number of internal nodes: 3

Enter your choice: 5
Minimum element: 34

14. Exit
Enter your choice: 10
Number of leaf nodes: 1
```

```
Enter your choice: 7
Enter the data to be searched: 34
Element 34 is found.
```

ERROR TABLE

S.No	Type of Error	Description
1	Syntax Error	Incorrect syntax while defining BST functions and statements.
2	Indentation Error	Improper indentation inside recursive functions and loops.
3	Attribute Error	Accessing left or right child attributes incorrectly.
4	Name Error	Using an undefined variable or function name.
5	Type Error	Comparing values of different data types.
6	Logical Error	Incorrect insertion, deletion, searching or traversal operations.

EXPLANATION

A Binary Search Tree (BST) is a non-linear data structure in which each node contains data, a left pointer and a right pointer.
In a BST, all elements in the left subtree are smaller than the root, and all elements in the right subtree are greater than the root.
The main operations are insertion, deletion and searching of elements.
Tree traversal is used to visit all nodes in a specific order, such as Inorder, Preorder and Postorder.
Inorder traversal gives the elements in ascending order, while Preorder and Postorder follow different visiting orders.

LAB-10

AIM: write a python programtoimplementanbinaryAVL tree with its operations

Program:

```
print("Manaswini-CSE25102")
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


def height(root):
    if root is None:
        return 0
    return root.height


def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)


def right_rotate(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

y.height = 1 + max(height(y.left), height(y.right))
x.height = 1 + max(height(x.left), height(x.right))

    return x
```

```
def left_rotate(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

 x.height = 1 + max(height(x.left), height(x.right))
y.height = 1 + max(height(y.left), height(y.right))

    return y

def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    else:
        return root

 root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    # LL case
    if b > 1 and data < root.left.data:
        return right_rotate(root)

    # RR case
    if b < -1 and data > root.right.data:
        return left_rotate(root)

    # LR case
    if b > 1 and data > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # RL case
    if b < -1 and data < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)
```

---

r⁹e⁰t u ᵖr⁵n⁸⁹root

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

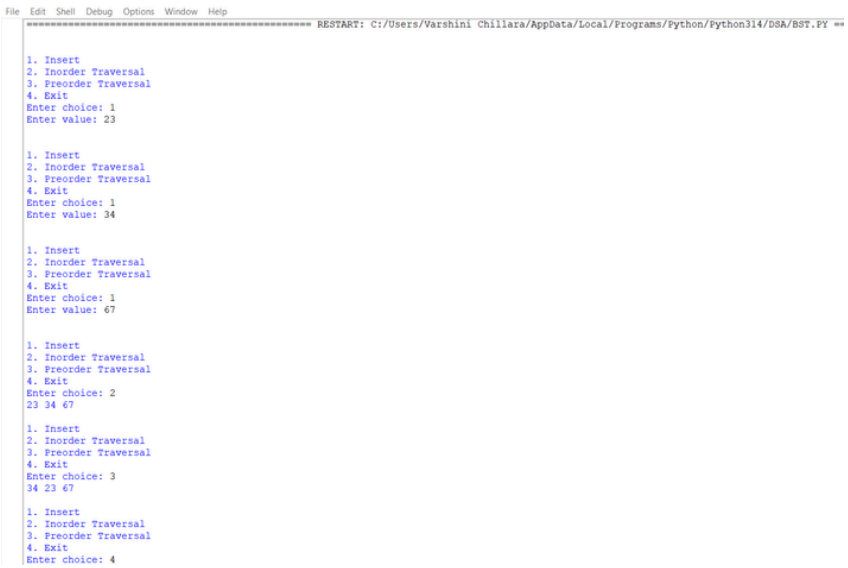

root = None

while True:
    print("\n\n1. Insert")
    print("2. Inorder Traversal")
    print("3. Preorder Traversal")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        inorder(root)
    elif ch == 3:
        preorder(root)
    elif ch == 4:
        break
    else:
        print("Invalid choice")
```

OUTPUT



Explanation:

AVLTree: Aself-balancing Binary Search Tree in which the balance factor of every node is -1, 0, or +1.

Insertion and Rotations: Insertion adds a node according to BST rules. Rotations (LL, RR, LR, RL) maintain the balance of the tree.

Traversal Operations: Inorder visits Left → Root → Right, Preorder visits Root → Left → Right, and Postorder visits Left → Right → Root.

Find Minimum and Maximum: The minimum value is found at the leftmost node, and the maximum value is found at the rightmost node.

Search Operation: Searches for a given value by comparing it with the current node and moving left or right accordingly.

Delete Operation: Removes the specified node, updates the heights, and performs rotations when necessary to maintain AVL tree balance.

ERROR TABLE

S.No	Type of Error	Description	Correction
1	Syntax Error	Incorrect quotation marks in the print statement.	Use `print("VARSHINI-CSE25105")`.
2	Indentation Error	Improper indentation inside functions and loops.	Maintain proper Python indentation.
3	Name Error	Using an undefined variable or function name.	Declare variables before using them.
4	Attribute Error	Accessing node attributes when the node is `None`.	Check whether the node exists before accessing its attributes.
5	Type Error	Comparing values of different data types.	Convert input values using `int()`.
6	Value Error	Entering non-numeric input for menu choice or node value.	Enter valid integer values.
7	Logical Error	Incorrect AVL tree rotation conditions.	Apply LL, RR, LR and RL rotations correctly.

LAB-11

1.Aim: Python program to implementHashing(Separatechaining, Linear probing, and Quadratic probing).

Program:

```
#Separate Chaining

size = 10

hash_table = [[] for i in range(size)]

def insert(key):

        index = key % size

        hash_table[index].append(key)

def display():

        for i in range(size):

                print(i, ":", hash_table[i])

#Main program

n = int(input("Enter number of elements: "))

        for i in range(n):

                key = int(input("Enter key: "))

                insert(key)

print("\nHash Table:")

display()
```

OUTPUT

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/hash.py =
Enter number of elements: 4
Enter key: 3
Enter key: 6
Enter key: 8
Enter key: 1

Hash Table:
0 : []
1 : [1]
2 : []
3 : [3]
4 : []
5 : []
6 : [6]
7 : []
8 : [8]
9 : []
```

Program:

```
#Linear Probing

size = 10

hash_table = [-1] * size

def insert(key):

  index = key % size


   while hash_table[index] != -1:
   index = (index + 1) % size


   hash_table[index] = key


def display():

  for i in range(size):

     print(i, ":", hash_table[i])

#Main program

n = int(input("Enter number of elements: "))

   for i in range(n):

       key = int(input("Enter key: "))

        insert(key)

print("\nHash Table:")

display()
```

OUTPUT

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/LINEAR PROBING.PY ==
Enter number of elements: 4
Enter key: 3
Enter key: 6
Enter key: 8
Enter key: 9

Hash Table:
0 : -1
1 : -1
2 : -1
3 : 3
4 : -1
5 : -1
6 : 6
7 : -1
8 : 8
9 : 9
```

Program:

```
Quadratic Probing

ize = 10

ash_table = [-1] * size

ef insert(key):

  index = key % size

  i= 0


 while i < size:
  new_index = (index + i * i) % size
 if hash_table[new_index] == -1:
       hash_table[new_index] = key
       return
   i += 1
 print("Hash Table is Full. Cannot insert", key)


ef display():

 for i in range(size):

    print(i, ":", hash_table[i])

Main program

 = int(input("Enter number of elements: "))

 for i in range(n):

    key = int(input("Enter key: "))

 insert(key)

rint("\nHash Table:")

isplay()
```

OUTPUT

```
========== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/Quadtaric probing.py =
Enter number of elements: 5
Enter key: 4
Enter key: 8
Enter key: 9
Enter key: 8
Enter key: 4

Hash Table:
0 : -1
1 : -1
2 : 8
3 : -1
4 : 4
5 : 4
6 : -1
7 : -1
8 : 8
9 : 9
```

EXPLANATION

Hashing: Hashing is a technique used to store and retrieve data quickly using a hash function.
Hash Function: index = key % size calculates the position where the key should be stored in the hash table.
Separate Chaining: If two keys get the same index, they are stored in the same index using a list.
Linear Probing: If a position is already occupied, the next available position is checked using (index + 1) % size.
Quadratic Probing: If a collision occurs, the next position is calculated using (index + i * i) % size.
Display Function: The display() function prints all the elements stored in the hash table.
Main Program: It accepts the number of elements and keys from the user and inserts them into the hash table.

ERROR TABLE

S.No	Error	Description
1	Syntax Error	Occurs when Python syntax rules are not followed.
2	Indentation Error	Occurs due to incorrect spacing or indentation in the program.
3	Name Error	Occurs when an undefined variable or function is used.
4	Type Error	Occurs when incompatible data types are used.
5	Index Error	Occurs when an invalid list index is accessed.
6	Value Error	Occurs when invalid input is given, such as entering a character instead of an integer.